PKM Kummer tower at P-224's 2-adic shape

The PKM Kummer tower at P-224's 2-adic shape, priced against matched rho

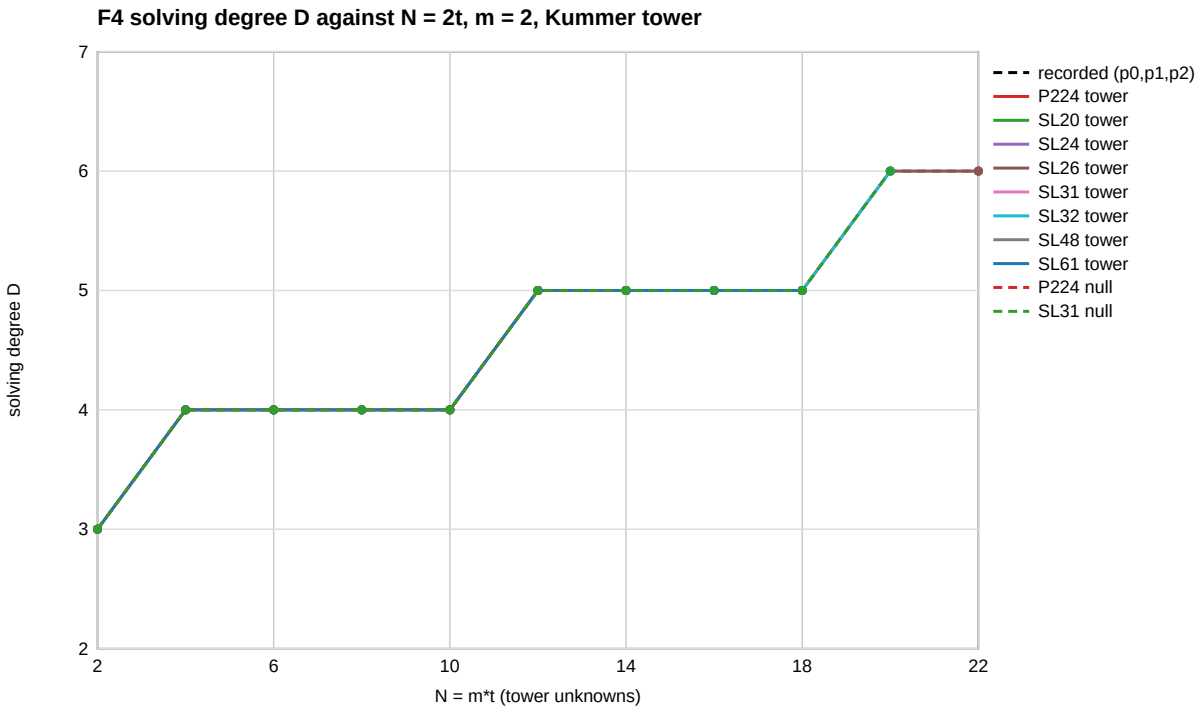
2026-10-22. Branch `claude/p224-pkm-kummer-10gpep`. Protocol: PROTOCOL.md, frozen in commit `375893d02` before any measured run; two dated amendments (arms, before any ecbench session; R18 and R18b, before R18b) are in the same file. Lead: claim 2d of research/p224_hypothesis_ranking_20261008/. Background: RESEARCH_PKM_TOWER_ORACLE.md §§10-17.

Every number is labelled **measured** (this directory's runs), **recorded** (an earlier committed row), **derived** (a calculation shown) or **predicted** (stated in the protocol before the runs).

Verdict

Inconclusive on the solver axis; clean negative on price; no improvement over the index-calculus baseline. Not an advance, not engineering, not relabelling. One accounting artefact is named in §3.4 so it is not mistaken for a gain.

- **Measured, solver axis (stage diagnostic, no S).** At eight P-224-shaped Solinas primes `p = 2^n - 2^k + 1` (20 to 61 bits) and at the NIST P-224 prime itself, the F4 solving degree `D` of the Kummer tower equals the recorded `D` in all 89 valued tower cells (`m = 2, 3, 4`), the width and step count of every system equal the recorded ones, and the shape-matched null gives the same `D` in all 65 paired cells. `k/n` ran from 0.42 to 0.70 and the odd part of `p - 1` from 6 to 128 bits; the prior pilot primes had `k/n` 0.87-0.94 and a 2-4 bit odd part. So the P-224 *proportions* of 2-adic structure change nothing F4 can see. H_same holds (predicted).
- **Measured, the one cell past the recorded frontier did not finish.** `m = 2, N = 24` (`t = 12`) was killed by the kernel's out-of-memory killer at 6.0 GB (R18, below), so it has no `D`. A capped re-run (R18b) gives only `D >= 6`. The `m = 2` line over `N = 2...22` reads **inconclusive** under rule A1 (plateau length 2, upper-half slope 0.114). Question Q2 stays open; it needs a host with more than this one's memory (its `B'` block passes 7.9e8 entries at step 26 of an unknown number).
- **Measured, price (ecbench, full cost, matched rho).** On four Solinas curves of 20 to 28 bits (and four Proth controls) the Kummer-base index calculus costs **18 to 78 times** rho with the negation map (`S` ratio, `w = 0`), **19.9 to 134 at `w = 1`**; rho is below every index-calculus arm on every curve at every weight of the band. Every one of the 1,152 runs (warm-ups included) verified and 864 replays reproduced.
- **Derived, the F4 method priced end to end at the one cell that matches a curve** (20 bits, `m = 2, N = 20`): at least **2.4 × 10^9** in `S`, about **1.9 × 10^9 times** matched rho at `w = 0` and **1.0 x 10^9** at `w = 1` (generous bound: 6 × 10^10 `S` with one field inversion costed as one multiplication). Only one F4 row exists, so there is no scaling fit for it.
- **Distance to rho (success criterion).** Best baseline-family arm: 18.4 times rho (20-bit Solinas, `ic-mitm`) and 75.9 times at 28 bits. At equal `t` the Solinas curves cost 2% to 32% more in absolute GAE than the prior ladder's Proth curves (§3.4), so no row moved toward rho.



Measured rows of runs/*.jsonl (pkm_p224_kummer_shape_20261022); recorded = PKM note 11-12, 16. Stage diagnostic, no S.

1. What ran

run	what	status (measured)
R0	positive control: isogeny kind, `p0`, `m = 2`, `N = 4-10`	**passed**: `D = 5` on both targets at every `N`
R1-R3	Kummer `m = 2`, `N = 2-20`, tower and null, SL26, SL31, SL32	complete, 41 rows each
R4	SL20, `m = 2`, `N = 16-20` (the 20-bit priced curve's prime)	complete
R5-R8	`m = 3` (`N = 6, 9, 12`) and `m = 4` (`N = 8, 12`), tower and chain null	complete
R9-R12	SL48 (Fp64) and SL61 (Fp64), `m = 2` to `N = 18`, `m = 3` to `N = 12`, `m = 4` `N = 8`	complete
R13-R15	P-224 (Fp224): `m = 2` to `N = 16` and `N = 18`, `m = 3` to `N = 12`, `m = 4` `N = 8`; wide builder `--verify`	complete, 23/23 verified in `BigUint`
R16	SL24, `m = 2`, `N = 20` and 22, one target each; null at `N = 22` (R16n)	complete
R17	SL31, `m = 3`, `N = 15`, one target	complete, `D = 7`
R19	SL26, `m = 2`, `N = 22`, one target	complete
R18	SL24, `m = 2`, **`N = 24`**, 2 threads, cap 1.8e9 entries	**killed (OOM), no row, no D** (§4)
R18b	the same system, `--max-nnz 6e8 --trace` (Amendment 2)	stopped at step 26, `D >= 6`
s20-s28	ecbench sessions, Solinas and Proth curve per size	4 sessions, all audits OK

Where `N = 22` was measured (SL24 and SL26, one target each), `D = 6`, the recorded value. The recorded `m = 2` frontier was `N = 22`; this round did not pass it.

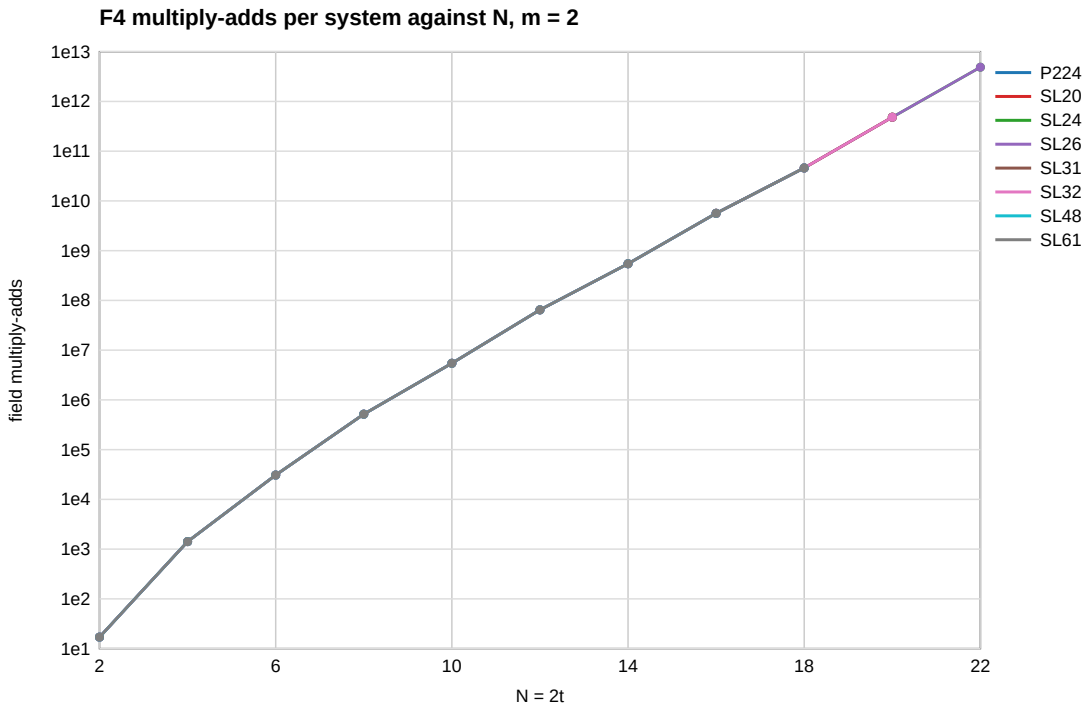
2. Solver axis (measured; stage diagnostic)

The tower is the Kummer tower `V = mu_{2^t}` (`g = 1`), `f4_tower_generic` at `Fp32`, `Fp64` and `Fp224`, two random targets per cell unless noted, seed 1347112276. Cell value: `D` when every target finished with the same `D`.

m	N	recorded D	SL20	SL24	SL26	SL31	SL32	SL48	SL61	P224	null D (same cell)
2	2	3	·	·	3	3	3	3	3	3	3 at every prime run
2	4	4	·	·	4	4	4	4	4	4	4
2	6	4	·	·	4	4	4	4	4	4	4
2	8	4	·	·	4	4	4	4	4	4	4
2	10	4	·	·	4	4	4	4	4	4	4
2	12	5	·	·	5	5	5	5	5	5	5
2	14	5	·	·	5	5	5	5	5	5	5
2	16	5	5	·	5	5	5	5	5	5	5
2	18	5	5	·	5	5	5	5	5	5	5
2	20	6	6	6	6	6	6	·	·	·	6 (SL26, SL31, SL32)
2	22	6	·	6	6	·	·	·	·	·	6 (SL24)
2	**24**	no value	·	**stopped, D ≥ 6**	·	·	·	·	·	·	·
3	6	5	·	·	5	5	5	5	5	5	5
3	9	6	·	·	6	6	6	6	6	6	6
3	12	6	·	·	6	6	6	6	6	6	6
3	15	7	·	·	·	7	·	·	·	·	not run
4	8	6	·	·	6	6	·	6	6	6	6
4	12	7	·	·	7	7	·	·	·	·	7

· = not run. Full per-cell table (targets, width, steps, multiply-adds, time): `tables/f4_cells.md` and `tables/f4_cells.tsv`; decision lines: `tables/f4_summary.txt`.

- **Q1, H_same: held.** 89 of 89 valued tower cells equal the recorded `D`; 0 differ. **Q3:** tower `D` equals null `D` in 65 of 65 paired cells (H1b does not hold: the tower is not below the null). **Q4:** at every `(m, N)` run at two or more primes (16 cells) the width and step count are equal, and the multiply-add count of target 0 agrees to within 3 × 10^-6 across the primes (`m = 2, N = 18`: 45,950,099,888 to 45,950,147,498 at seven primes from 20 to 224 bits; one thread, so the chunking is the same).
- **Positive control (R0): passed**, so the instrument can see a tower-family difference (isogeny `D = 5` against Kummer 4 at the same `N`).
- **Independent check.** `pkm_tower_check verify` recounted every finished `u64`-builder tower row by exhaustive search over `V^m`: 104 + 52 rows (R0 included), all agree (`verify_u64.txt`, `verify_u64_wide48_61.txt`). The P-224 rows (above `2^64`) were checked by the pilot's own `--verify` in `BigUint` arithmetic: 23 of 23 tower rows ok (the independent checker's integer type does not hold `p`, so it does not read those rows).
- **Q2 (frontier).** Pooling the `m = 2` tower rows over primes (a derived input file, `analysis/pooled_m2_kummer_tower.jsonl`, `p` relabelled to 1, `target_index` offset per prime, nothing else changed) and running `pkm_tower_check analyze` gives for `N = 2...22`: *A1 inconclusive (upper-half slope 0.114 over N = 12...22, final plateau L = 2)* (`analysis/pooled_m2_analyze.txt`). R18b's trace, `runs/R18b-SL24-m2-N24-sizing.log`: 25 steps ran, step 23 was at degree 6 and added 2,042 elements, so **`D >= 6`**. Step 26 (degree 6) stopped at `B'` = 7.86e8 entries, above the 6e8 cap. The recorded `N = 22` system had 28 steps in all. Nothing here reads 7.



Measured; mean over the cell's finished targets. Counts depend on the thread count (chunking).

Scope. `t <= 12` at `N <= 24`. P-224's `p - 1` has `v2 = 96`, so a tower could in principle be `t = 96`. PKM needs `|V|^m` of the order of `r`, that is `N = m t` near `log2 r = 224` at any `m`. The measured `N <= 24` is about 11% of that, and every deeper cell is extrapolation (not made here).

3. Price (measured arms; derived F4 arm)

3.1 Boundary, reference, unit

`S = total GAE / sqrt(r)`; the floor is `sqrt(pi/4) = 0.886`; the reference is `rho.negation` in the same session and binary, matched to each IC arm's workload and round. All 8 workloads per curve are one-target; 3 measured rounds; every IC arm charges every group operation.

3.2 Curves

Curves are named here by their field and parameters, not by an ICV1 slug (see the note below the table). `y^2 = x^3 + ax + b`, `#E = h r`, `r` prime.

bits	Solinas curve `(p, a, b)`	k	h	log2 r	t	Proth control `(p, a, b)` (prior ladder)	log2 r
20	(1032193 = 2^20-2^14+1, 115752, 453155)	14	1	19.98	8	(946177, 304973, 530845)	18.85
24	(16760833 = 2^24-2^14+1, 10096653, 5158538)	14	4	22.00	9	(10940417, 10128206, 867458)	23.38
26	(67104769 = 2^26-2^12+1, 36772902, 59108565)	12	2	25.00	10	(40857601, 8702713, 9617118)	23.28
28	(268369921 = 2^28-2^16+1, 147431093, 254493867)	16	1	28.00	11	(231657473, 95419558, 159693275)	26.20

Not registered. AGENTS.md §11 asks for registration before a curve's slug is cited. The four Solinas curves are not in `docs/curves/registry.json`: a registry change moves the registry digest that the cover, trait and transfer certificates bind, so it needs those regenerated

(curve_cover_check, curve_traits, jacobian_transfers, the alias map), which is a separate, large change. So no slug is written in Markdown here (the slugs are in the sealed sessions' plan.json, spec.json and records, and ecbench plan says registered=false for them). Registering them is a follow-up. The four Proth controls are registered (the prior ladder).

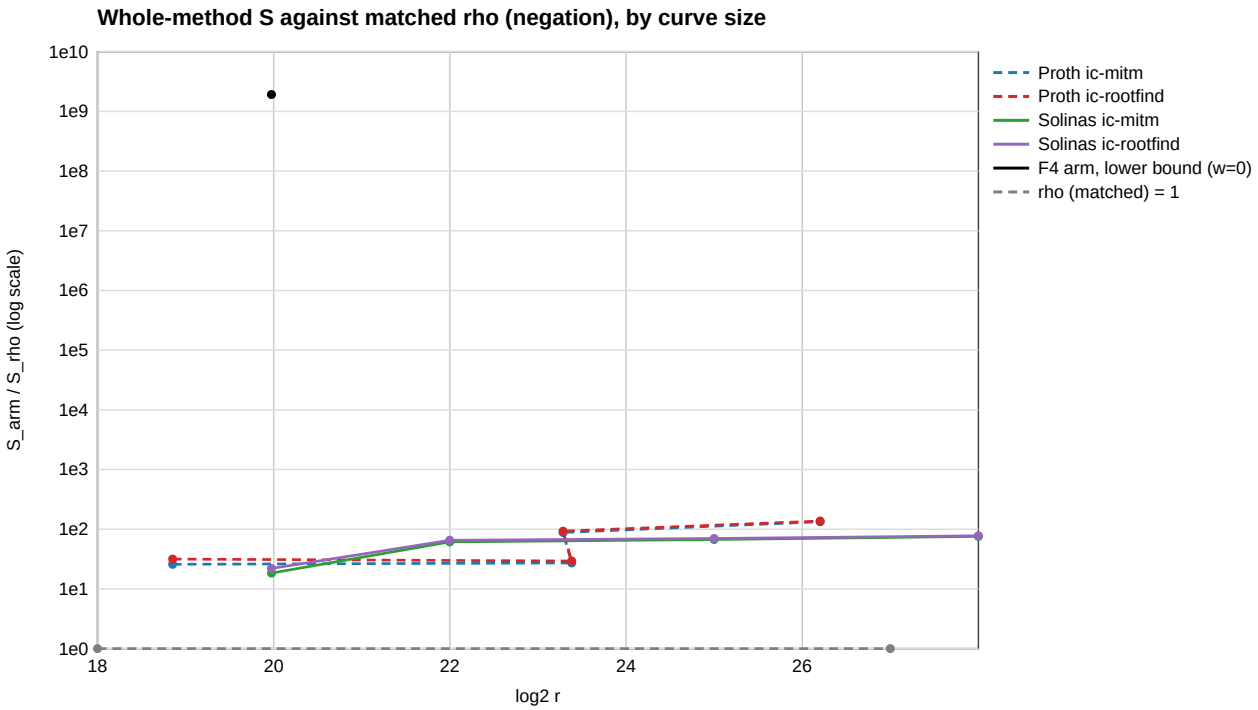
3.3 Table: S per arm, ratio to matched rho, with the §5a band

S at w = 0 / 0.1 / 1 GAE per unpriced native operation (the harness's ecbench band, reproduced by tables/ecbench_arms.md); ratio = S_arm / S_rho at the same w. The F4 arm is **derived**, one row, a lower bound (§3.5). D is the F4 solving degree of the cell the arm's size corresponds to (stage diagnostic).

curve	arm	D (stage)	S w=0 / 0.1 / 1	ratio to rho w=0 / 0.1 / 1	unpriced counters (mean)
20-bit Solinas	rho-neg		1.250 / 1.358 / 2.337	1.0	canonicalisations 1,104
20-bit Solinas	ic-mitm		22.94 / 25.30 / 46.53	18.4 / 18.6 / 19.9	lookups 6,092; row ops 17,357; 3 more
20-bit Solinas	ic-rootfind		27.54 / 29.96 / 51.69	22.0 / 22.1 / 22.1	lookups 6,167; row ops 17,845; 3 more
20-bit Solinas	**F4 arm (derived), m=2, N=20**	6	2.40e9 / 2.40e9 / 2.40e9	1.9e9 / 1.8e9 / 1.0e9	F4 pairs, reducers, rows (§3.5)
24-bit Solinas	rho-neg		1.057 / 1.155 / 2.030	1.0	
24-bit Solinas	ic-mitm		64.9 / 73.8 / 153.7	61.4 / 63.9 / 75.7	
24-bit Solinas	ic-rootfind		68.9 / 77.7 / 157.7	65.1 / 67.3 / 77.7	
24-bit Solinas	F4 arm	not priced: N = 24 has no D			
26-bit Solinas	rho-neg		0.913 / 0.998 / 1.762	1.0	
26-bit Solinas	ic-mitm		61.2 / 74.6 / 194.8	67.1 / 74.7 / 110.6	
26-bit Solinas	ic-rootfind		63.8 / 77.1 / 196.4	69.9 / 77.3 / 111.4	
28-bit Solinas	rho-neg		0.923 / 1.013 / 1.824	1.0	
28-bit Solinas	ic-mitm		70.1 / 87.5 / 244.0	75.9 / 86.3 / 133.8	
28-bit Solinas	ic-rootfind		71.8 / 89.0 / 243.3	77.8 / 87.8 / 133.4	
20-28-bit Proth controls	rho-neg, ic-mitm, ic-rootfind		see tables/ecbench_arms.md	25.9-133.7 (w=0)	

Paired ecbench compare intervals (95% cluster bootstrap over 8 workloads, per curve, w = 0), ic-mitm / rho-neg: 18.4 [14.7, 23.9] (20-bit Solinas), 61.4 [49.9, 78.0] (24), 67.0 [53.1, 86.4] (26), 75.9 [60.5, 97.4] (28); ic-rootfind / rho-neg: 22.0 [17.6, 28.9], 65.1 [52.9, 82.7], 69.9 [55.4, 90.1], 77.8 [62.0, 99.8]. The Proth controls: 25.9, 27.2, 88.2, 133.7 (mitm). A/A (rho-neg-aa / rho-neg) = 1.0000 on all four sessions. Files: tables/s*.compare.*.txt.

Order across the band. `rho-neg < ic-mitm` and `rho-neg < ic-rootfind` hold at every `w` in {0, 0.1, 1} on all eight curves; the only unresolved pairs are `rho-neg` against its own A/A copy and `ic-mitm` against `ic-rootfind` on the 28-bit Solinas curve (`tables/s*.band.txt`). The F4 arm's order against rho holds at every `w` (its `S` is `w`-independent to 4 digits: `U` = 213,652 unpriced operations per system is 1.1e8 GAE at `w = 1` over 522 relations, against 2.4e12 GAE priced).



ecbench sessions (measured) and the derived F4 lower bound (PROTOCOL 4.4). Solid: Solinas, dashed: Proth.

3.4 Solinas against Proth at the same tower height

Predicted (P6): `S_IC/S_rho` on a Solinas curve within 20% of the Proth control at each size. **Failed as stated**: the ratios differ by -29%, +126%, -24% and -43% at 20, 24, 26, 28 bits (18.4 vs 25.9, 61.4 vs 27.2, 67.0 vs 88.2, 75.9 vs 133.7). The differences are not the prime's shape:

- `t` is chosen by bit size, so a Solinas and a Proth curve of the same size have the same factor-base height, while `r` differs (cofactors 1-4: `log2 r` 19.98 vs 18.85, 22.00 vs 23.38, 25.00 vs 23.28, 28.00 vs 26.20). `S` divides by `sqrt r`.
- **Absolute cost at equal `t`** (derived from the sealed records, 95% bootstrap over workloads, `tables/gae_solinas_vs_proth.md`): Solinas / Proth GAE for `ic-mitm` is 1.053 [1.034, 1.071], 1.324 [1.297, 1.357], 1.206 [1.196, 1.216], 1.017 [1.014, 1.020] at 20, 24, 26, 28 bits, and for `ic-rootfind` 1.034, 1.300, 1.197, 1.016 (same intervals' order). **The Solinas curves cost more in absolute GAE at every size**, 2% to 32%, not less, in both IC arms. Rho moves with `sqrt r` (Solinas/Proth rho GAE 1.48, 0.59, 1.59, 1.79).
- So where the Solinas `S` ratio is lower, rho got more expensive, the index calculus did not get cheaper. By AGENTS.md §3 that is **accounting** (the numbers changed, the algorithm did not) and it is not claimed as an improvement. No row met the protocol's improvement condition (§5 of the protocol) once `r` is accounted for.
- Growth with size (`d log2(S_ic/S_rho) / d log2 r`, 4 sizes, least squares, 95%): Solinas `ic-mitm` 0.217 ± 0.225, `ic-rootfind` 0.192 ± 0.202; Proth `ic-mitm` 0.304 ± 0.350, `ic-rootfind` 0.268 ± 0.347; prediction 0.17, no fit excludes it or zero (`tables/growth_fits.txt`). With `t` tied to `bits` and `r` varying by cofactor, this fit is poorly conditioned; it is reported, not relied on.

3.5 The F4 arm (derived; PROTOCOL §4.4, formula frozen before the runs)

`GAE_F4 = R * (madds / M + w * U)`, `R = columns + 10`, `S = GAE_F4 / sqrt r`.

input	value	label
curve	20-bit Solinas, `r = 1,030,723`, `t = 10` (`N = 20`)	
`columns`	512, the on-curve abscissae of `ic-mitm-tf4` (`prime-kummer:t=10`)	measured (factor-base record)
`madds`	4.819e11 per system, mean of 2 random targets (R4)	measured
`ns / madd`	1.05, `ms / madds` of R4 under load	measured
`ns / GAE`	101-139 across five invocations of the report tool; the table uses the stored one, 108.0 (microbenchmark of the harness's counted `PrimeCurve` addition, median of 5 runs of 10^6 additions; `analysis/ns_per_gae_1032193.txt`)	measured
`M = ns/GAE / ns/madd`	97-133 madds per GAE (the table uses the stored one: 103.2)	derived
`U`	213,652: F4 pairs + reducer rows + max rows per system	measured, priced at `w`
`S_F4`	2.40e9 at every `w`; ratio to rho 1.9e9 / 1.8e9 / 1.0e9	derived, **lower bound**
generous bound	`M = 4` (inversion costed as one multiplication): `S_F4 = 6.2e10`	derived

Lower bound because it counts one F4 solve per relation, 522 relations (`R`); it omits the failed targets, the factor-base build and the linear algebra. A second priced row needs `N = 24` (R18 failed), so there is **no scaling fit** for the F4 arm. Per the protocol the 26- and 28-bit curves, which need `N = 26, 28`, get no F4 row and none is extrapolated. `tables/f4_priced.md`.

Prediction P7 (F4 arm at least 10^5 times rho at `N = 20`): **held** (1.9 × 10^9).

4. Failures and deviations (all preserved)

1. **R18 was OOM-killed (measured, kernel log).** SL24, `m = 2`, `N = 24`, one target, 2 threads, `--max-nnz 1.8e9`, `ulimit -v` 9 GB. Started 04:19 UTC; killed about 06:45 UTC by the memory-cgroup OOM killer, `anon-rss 6,007,580 kB`, the killer having been invoked by another agent's process in the shared container cgroup. The pilot writes a system's row when the system ends, so the run left no row and no step trace: no `D` and no lower bound. Per protocol §2.2 it is recorded as out of budget (memory) and is **not re-run**. The empty files `runs/R18-SL24-m2-N24.jsonl` and `.log` are kept.
2. **R18b (Amendment 2, committed before it ran)** re-ran the same system with a 600M-entry cap and a trace. It stopped by design at step 26 and gave `D >= 6` only (§2).
3. **`progress.txt` exit statuses are void for `run_f4.sh`.** The `echo` printed the `$?` of its own `date` substitution, so every `end ... exit 0` line from that script carries no information (R18 shows `exit 0` though the kernel killed it). Completion is read from each run's summary line in `runs/*.log` (the only runs without one: R18). The script is fixed in this tree; the version that ran had the bug.
4. **Parallel runs.** The main script was stopped by hand after R16 started (the child kept running), and R16n, R17 and R19 were started in parallel with the same flags through `run_parallel.sh` to use idle cores; `progress.txt` has a NOTE (the script was tidied of a redundant `case` after the runs, same flags). The cells and flags are the protocol's; only the order and concurrency changed.

5. **Load.** The host's load average was 4-10 throughout, shared with other agents, so wall times (`ms`) are descriptive only; operation counts do not depend on it.
6. **ecbench ran at L0**, `--cpus none --allow-busy`, so no wall time is admitted; operation counts, audits and replays are unaffected. The spec says `isolation_required: L2`; the harness ran and graded every run L0.
7. **Scratch smoke runs.** Before the first run of each tool the author ran short smoke tests of the pilot at SL26 and P-224 (`N <= 6`, 14 rows) in a scratch directory; they are not results and are not in this directory.
8. **A transcription slip in the report tool**: the recorded `D` at `N = 10` was first coded as 5; the protocol says 4, and three cells were reported as `H_diff` in a trial print before the fix. The committed output is from the fixed tool; no run changed.
9. **Curves not registered** (Amendment 3 in the protocol; §3.2 says why).

5. Predictions against outcomes

	prediction	outcome
P1	R0 gives `D = 5`, `N = 4-10`	**held**
P2	H_same on every valued cell; D rises to 6 at `N = 20` at every prime	**held** (89 of 89)
P3	`D(N = 24)` is 6 or 7	**no value** (R18 killed; R18b `D >= 6`)
P4	tower and null agree at every cell	**held** (65 of 65)
P5	multiply-adds equal across backends at one thread	**held** (to 3 × 10^-6)
P6	Solinas S ratio within 20% of Proth	**failed as stated**, explained by `r` (§3.4)
P7	F4 arm at least 10^5 x rho	**held**

6. What this shows, what it does not, and the next step

Shows (measured). The F4 solving degree and the whole F4 computation do not see a P-224-like ratio of 2-adic to odd part: equal `D`, width, steps and multiply-adds at eight Solinas primes and at P-224's. The tower family still reads inconclusive at `N <= 22`. Priced end to end the PKM F4 method costs 10^9 times rho at the one size that matches a curve, and the table oracles on the same base cost 18-134 times rho and the gap grows slowly with size.

Does not show. Anything at `N` near P-224's size, `m >= 4` at `N` above 12, whether `D` rises past 6 at `m = 2`, or a priced F4 row at a second size. Nothing here attacks P-224; a bounded null is not nonexistence (AGENTS.md).

Next improvement to try (each is a different experiment, not done here):

1. `m = 2, N = 24` and `m = 3, N = 21` on a host with 64 GB or more (`B'` is 4 bytes per entry; R18b's step 26 alone is 3.1 GB, and the run died at 6.0 GB resident), which would answer Q2 and give the F4 arm a second priced size.
2. An `ic.pipeline` oracle that calls `f4_tower_generic`, so the F4 arm is measured in the harness (`ecbench-extend`) instead of derived.
3. Curves matched on `r`, not on bits, for any Solinas-against-Proth `S` comparison, since §3.4 shows the bit-matched one is confounded.

7. Environment and reproduce

Claude Code cloud container, 4 vCPU Xeon @ 2.10 GHz, 15 GiB RAM shared with other agents, rustc 1.97.0, binary `pkm_tower_pilot` sha256 `aab76a30ab352de3…` (runs/BINARY.sha256), built from commit `dd0486c6d` (the pilot source is unchanged from `main`). Conductor `check` reported the scope clear.

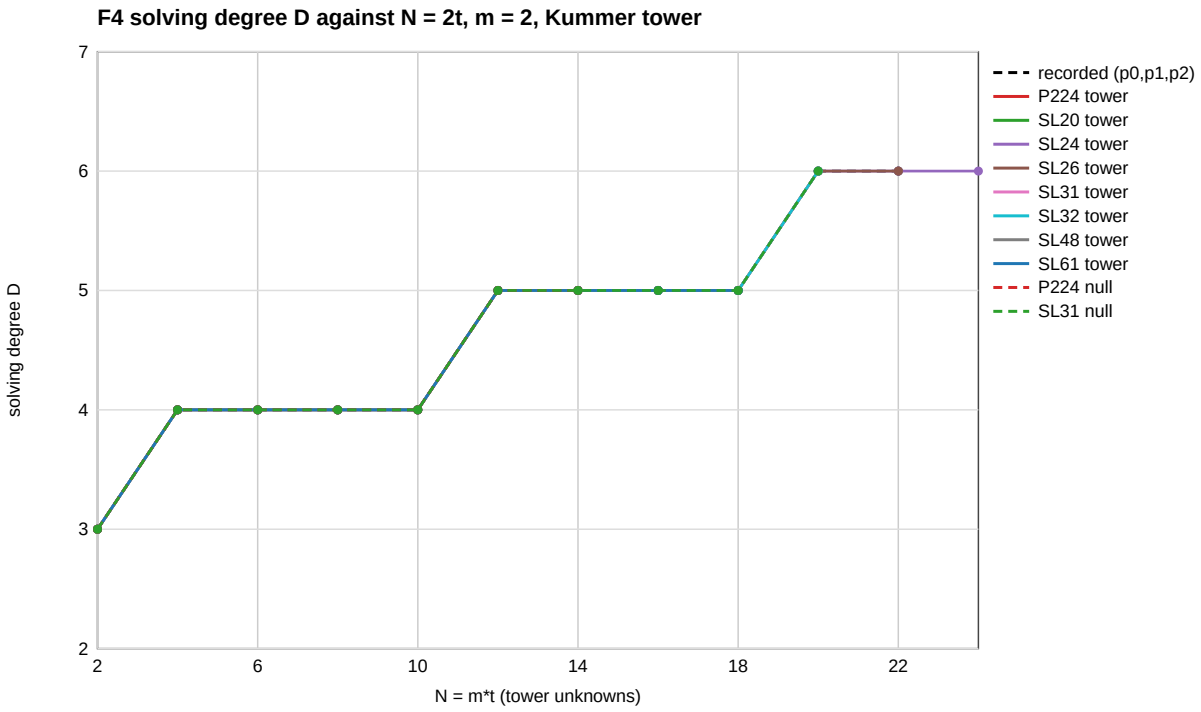
```
        export CARGO_TARGET_DIR=/path/to/target
        cargo build --release --example pkm_tower_pilot --example pkm_tower_check \
          --example prime_tower_curves --example pkm_p224_shape_report --bin ecbench
        research/pkm_p224_kummer_shape_20261022/make_specs.sh          # specs from
curves.jsonl
        BIN=$CARGO_TARGET_DIR/release/examples/pkm_tower_pilot
research/pkm_p224_kummer_shape_20261022/run_f4.sh main
        BIN=... research/pkm_p224_kummer_shape_20261022/run_f4.sh frontier   # R18 (needs
~10 GB)
        BIN=... research/pkm_p224_kummer_shape_20261022/run_r18b.sh
        ECBENCH=$CARGO_TARGET_DIR/release/ecbench
research/pkm_p224_kummer_shape_20261022/run_sessions.sh
        pkm_p224_shape_report f4 runs .  &&  pkm_p224_shape_report price runs . curves.jsonl
sessions/s20 sessions/s24 sessions/s26 sessions/s28
        research/pkm_p224_kummer_shape_20261022/render.sh              # PNGs and report.pdf
```

Curves: `prime_tower_curves --solinas 20:14,24:14,26:12,28:16 --seed 20261022 --max-cofactor 4` (curves.jsonl, curves.log). Specs: specs/. Sealed sessions: sessions/; audits (audits/): s20 and s24 320 records, 240 replays; s26 and s28 256 records, 192 replays; every record verified, every replay reproduced, 0 problems. Run rows and logs: runs/. Report PDF: report.pdf.

8. Addendum 2026-10-22: slugs, the R18 memory fault, and the `N = 24` rerun

The sections above are frozen and unchanged. This addendum answers the two follow-ups they listed (§3.2 "Not registered"; §6 item 1 and §4 item 1: the `N = 24` cell and the R18 kill). Its protocol is PROTOCOL.md §12 (Amendment 4), committed before any follow-up run; its runs, scripts and tables are in followup_20261022/. Labels: **measured**, **recorded** (an earlier committed row), **derived**, **predicted**.

Verdict of the addendum. The four Solinas curves are registered and the catalog files derived from the registry are regenerated. R18 died because its footprint (one dense block of 5.4 GB at step 27, 6.0 GB resident at the kill) was asked of a shared host with no byte bound and no per-step output; the fix bounds and records it and returns about 1 GB between steps, but **does not lower the peak**, which is the block itself. With the host's memory free, the `N = 24` cell finished: **`D = 6`**, the same as at `N = 20` and `22`, so the `m = 2` line is still **inconclusive** under A1 (plateau `N = 20...24`). The F4 arm priced at the 24-bit curve is about **7.7 × 10^10 times** matched rho. No row moved toward rho; this is engineering and diagnosis, not an improvement of the index-calculus baseline.



Measured rows of runs/*.jsonl (pkm_p224_kummer_shape_20261022); recorded = PKM note 11-12, 16. Stage diagnostic, no S.

8.1 Registration (follow-up (a))

bits	ICV1 slug	EC1	p	#E
20	`icv1-fp20-t1471-592e3734`	`EC1P20Cfph889896aa38c5`	1032193	1030723
24	`icv1-fp24-tm3090-db44b2b1`	`EC1P24Cfphe9a49c9b849b`	16760833	16763924
26	`icv1-fp26-t5908-23ba8088`	`EC1P26Cfph5c2cfa4c3fe4`	67104769	67098862
28	`icv1-fp28-tm7557-9dd7f10c`	`EC1P28Cfphe9541bad23ac`	268369921	268377479

These are the slugs the sealed sessions already carried. They are registered through a new source record, `docs/curves/sources/pkm-p224-shape-20261022.json`, so §3.2's "Not registered" paragraph is superseded (its text is left as it was).

How, and what was not done. `scripts/build_curve_registry.py` run on the tracked tree would add 5,459 curves besides these four (it finds them in tracked research JSON; the committed registry never took them up), so a full regeneration would be a different, much larger change. The four entries the generator produces were spliced into the committed registry in the generator's own sort order and the alias map regenerated from the result; nothing else in the registry moved. Then every derived file was regenerated with its own tool and its check run: `curve_cover_check` (covers, cover links: 2,576 verified), `curve_traits build` and `check`, `jacobian_transfers build` and `check`, `update_curve_standards.py` (IC leaderboard roster), `build_lab_browser.py`, `build_cover_transfer_assessments.mjs --write` and `--check`, the assessment verifier (`verify_assessment.mjs`: 0 problems) and its schema validation, and the WeakCurves snapshot package (snapshot, GHS audit, property export, isogeny graph: each differs from before only in digest lines). Tests that read the catalog passed (`cargo test --release` on `curve_cover_check`, `curve_standards`, `hyperelliptic`, `binary_ecc::cover_transfer`, `hyperelliptic-cover`, `curve_traits`, `jacobian_transfer`; the Python identity and cover-catalog tests; `check_curve_names.py`).

Things the registry change forced, stated so they are not mistaken for results: the assessment builder and verifier pin the catalog size, so their counts moved by four (2,576 maps, 2,574 unique covers, 981 EC1 representations, 513 declared fields, 1,057 direct odd prime rows, 2,411 odd rows) and two semantic digests whose text carries those numerals were re-pinned after checking the only differences were the numerals; the assessment is pinned to the commit that registers the curves (`228eec9a9`); its report, graphs and PDF were regenerated (PDF with XeLaTeX, as Tectonic is not installed here); the producer-source digest in `HYPERELLIPTIC_INFRASTRUCTURE.md` follows a one-line test-constant change. **Not done:** three site copies (`website/weakcurves/dist/{snapshot,ghs_genus128_cost_audit,isogeny_graph}.json`, the WeakCurves README's copy step) were refused by the permission layer, so the package's `validate_attack_map.py` equality check fails until they are copied from `research/weak_curve_leaderboard_20261012/`; and its append-only `ARTIFACTS.sha256` was already failing on three of its Markdown files before this change and was not touched.

8.2 Why R18 died (follow-up (b)), from recorded evidence

1. `B'`, the reduced reducer block, is dense, 4 bytes an entry. R18b's stop line recorded 785,810,929 entries at step 26 (3.14 GB). The rerun measured the rest of the system (§8.4): step 27 holds **1,357,804,954 entries (5.4 GB)**. R18's `anon-rss` at the kill, 6,007,580 kB (5,867 MB), is of the size of that step (the rerun's peak resident set there is 5,991 MB): R18 was inside step 27, killed near its own peak need, not by a leak.
2. R18 ran with `--max-nnz 1.8e9`, which the engine also applies to `B'` when `--max-dense` is not given (`max_dense.or(max_nnz)`): up to 7.2 GB of `B'` was allowed, against a 15 GB host shared with other processes. `ulimit -v` caps address space, not what the host has free. There was no byte bound.
3. After step 25, R18b's stop line had 1,322 MB resident but 172 MB live heap: about 1.1 GB of freed memory sat in the allocator's per-thread arenas.
4. No `--trace`, and a row is written when a system ends: the kill left nothing.

8.3 The fix, and what it did not change

On the generic engine (every committed F4 row): `B'` in one slab instead of one `Vec` per row; `max_bytes` (`--max-mem-mb`), a bound on a step's dense blocks checked before they are allocated; `malloc_trim` after each step; `--steps-out`, one flushed JSON line per step; a per-step non-zero count of `B'` (both engines). Not built: compressing or spilling `B'`.

Checks that what is measured did not change (all passed; pre-registered, E1-E3):

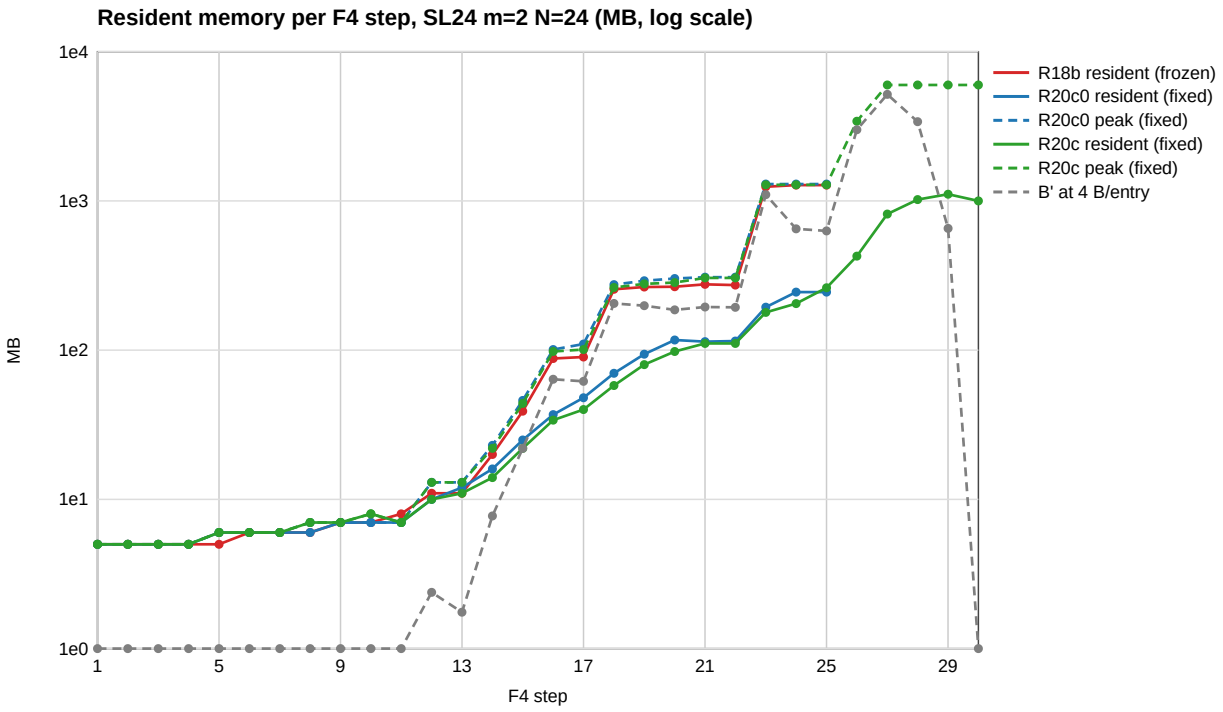
check	against	result
E1: SL20 `N = 16, 18, 20` (6 rows) and SL24 `N = 20`, 1 thread	frozen R4, R16 rows	7 of 7 rows equal in every field but the wall clock (`pkm_tower_check compare-rows`, and `check_rows.sh`)
E2: SL24 `N = 20`, 22, 2 threads, fixed against pre-fix binary	each other	equal rows (`N = 22`: 4,877,824,907,495 multiply-adds both)
E3: SL24 `N = 24`, R18b's flags, steps 1-25 and the stop line	R18b's trace	25 of 25 steps identical (`compare-trace`); the stop line equal (`B'` 785,810,929, nnz 499,596,237)
R20c steps 1-25 at 4 threads	R18b's trace (2 threads)	25 of 25 identical: the count did not depend on 2 against 4 threads (it does depend on 1 against 2: `N = 20` 481,871,455,568 against 481,961,794,016)

`tables/equality_checks.txt`.

Memory (E2, measured; one run per arm on a shared host). Peak resident set (`VmHWM`) and what stays resident after the last step:

cell	arm	peak (MB)	after the last step (MB)	multiply-adds
`N = 20`, 2 threads	pre-fix	466	457	481,961,794,016
`N = 20`	fixed	521	175	same
`N = 20`	fixed, `MALLOC_ARENA_MAX=2`	497	133	same
`N = 22`, 2 threads	pre-fix	1,536	1,435	4,877,824,907,495
`N = 22`	fixed	1,499	332	same

So the fix returns memory between steps (R18b held 1,245-1,277 MB from step 23; the fixed run 179-262 MB over the same steps) and **does not lower the peak**: -2.4% at `N = 22`, +12% at `N = 20` (the slab is allocated whole). **P9 failed** (predicted at most 0.90 of the baseline's peak). `tables/e2_memory.md`.



Recorded: R18b trace (frozen binary) and followup_20261022 steps files (fixed binary). One run each, shared host.

`B'` is not compressed. Its non-zero share is 41% at step 23, 35% at step 26, 57% at the largest step (27), and 77% and 96% after. A bitmap with packed values would cut `B'` by about half only where it is under half full (steps 23-26), not at the largest step; and any encoding that skips zeros turns the dense AVX2/AVX-512 axpy kernels into scalar scatters, which this round did not measure. **P11** (at least half of `B'` non-zero at the largest steps) holds at the largest step and fails at steps 23 and 26.

8.4 The rerun, R20c (SL24, `m = 2`, `N = 24`, target 0): finished

Flags as the frozen runs, 4 threads, `--max-mem-mb 7800`, `MALLOC_ARENA_MAX=2`, `nice`; host `MemAvailable` 13.7 GB at the start (the start rule needed 10 GB); started 21:34:17 UTC.

quantity	measured
outcome	finished at step 30: the target's system is inconsistent (refuted), `D = 6`, `max_cols_to_solution` 103,785
wall	7,245 s (2.0 h), shared host, 4 threads

quantity	measured
multiply-adds	4.2213 × 10^13 (predicted 4 × 10^13 to 5 × 10^13)
largest matrix	1,618,266,163 non-zeros (predicted 1.3 × 10^9 to 1.8 × 10^9, PROTOCOL §2.4)
largest B'	1,357,804,954 entries, step 27 (1.36 times my derived estimate of 1.0 x 10^9, PROTOCOL §12.1)
peak resident	5,991 MB (VmHWM), 5,956 MB sampled every 5 s; the byte bound (7,800) never bound
pairs, rows	354,247 critical + 98,720 tower pairs; 285,855 reducer rows; 186,749 rows at most

Steps 23-30 (all 30 in tables/R20c_steps.md):

step	degree	new elements	cols	B' entries	non-zero	multiply-adds	s	resident after / peak (MB)
23	6	2,042	44,562	289,092,933	41%	8.6e11	251	179 / 1,281
24	5	953	29,248	170,382,738	43%	1.2e11	27	205 / 1,281
25	5	222	29,028	165,128,367	49%	8.1e10	20	262 / 1,281
26	6	4,189	67,939	785,810,929	35%	6.0e12	1,822	427 / 3,422
27	6	5,797	103,785	1,357,804,954	57%	1.5e13	2,687	816 / 5,991
28	6	7,364	91,528	891,092,020	77%	1.6e13	1,978	1,022 / 5,991
29	5	3,923	47,424	171,739,455	96%	4.1e12	414	1,110 / 5,991
30	3	37	2,325	84,656	100%	2.3e7	0	1,000 / 5,991

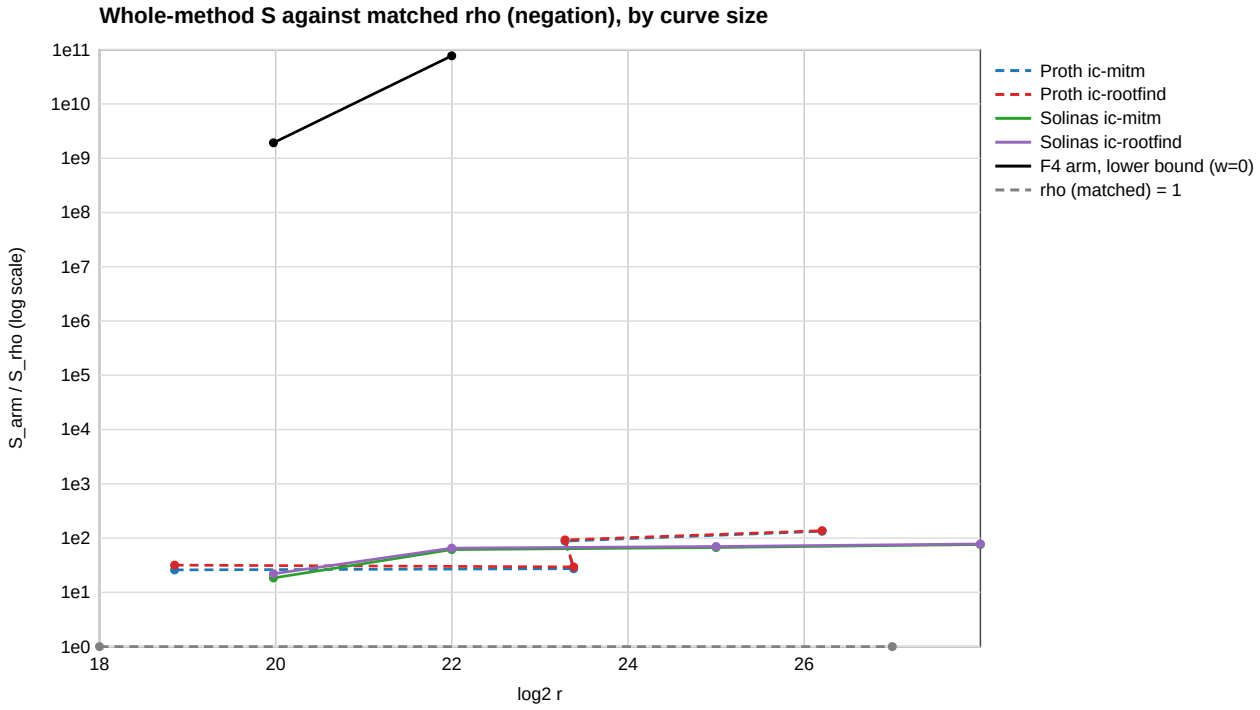
Q2. Pooling `m = 2` as in §2 with this row (70 systems; followup_20261022/analysis/pooled_m2_analyze.txt): `D = 6` at `N = 20, 22, 24`; final plateau `L = 4`; upper-half slope 0.129 over `N = 14...24`; **A1: inconclusive**. R18b's `D >= 6` is now `D = 6`. H_same: 89 of 89 cells with a recorded value still equal (`N = 24` has none, so it is a 90th valued cell); tower against null (65 pairs) unchanged (tables/f4_summary.txt). `m = 3, N = 21` was not run.

8.5 Price and distance to matched rho (success criterion: distance, not a gate)

Frozen ecbench rows are **recorded** (this directory's sessions, unchanged); the F4 rows are **derived** with the frozen formula (PROTOCOL §4.4, `R` from the s24 factor-base record). `S` at `w = 0 / 0.1 / 1` GAE per unpriced native operation; the ratio is to `rho.negation` of the same session at the same `w`. The order against rho holds at every `w` for every row (the only unresolved pairs are as in §3.3).

curve	arm	S w=0 / 0.1 / 1	ratio to matched rho w=0 / 0.1 / 1	label
20-bit	ic-mitm	22.94 / 25.30 / 46.53	18.4 / 18.6 / 19.9	recorded
20-bit	ic-rootfind	27.54 / 29.96 / 51.69	22.0 / 22.1 / 22.1	recorded
20-bit	F4, N = 20	2.40e9 (all w)	1.9e9 / 1.8e9 / 1.0e9	derived, lower bound
24-bit	ic-mitm	64.9 / 73.8 / 153.7	61.4 / 63.9 / 75.7	recorded
24-bit	ic-rootfind	68.9 / 77.7 / 157.7	65.1 / 67.3 / 77.7	recorded
24-bit	**F4, N = 24 (new)**	8.15e10 (all w)	**7.7e10 / 7.1e10 / 4.0e10**	derived, lower bound
26-bit	ic-mitm, ic-rootfind	61.2 / 74.6 / 194.8; 63.8 / 77.1 / 196.4	67.1 / 74.7 / 110.6; 69.9 / 77.3 / 111.4	recorded
28-bit	ic-mitm, ic-rootfind	70.1 / 87.5 / 244.0; 71.8 / 89.0 / 243.3	75.9 / 86.3 / 133.8; 77.8 / 87.8 / 133.4	recorded

The 24-bit F4 row (`R` = 2,045 relations, the factor-base record's 2,035 columns plus 10; `M` = 517.6 multiply-adds per GAE) uses the frozen conversion, ns per multiply-add = wall / multiply-adds of the cell, which here is a **4-thread** wall time (0.172 ns), so it understates the cost against a single-thread GAE; with the 1-thread conversion of R20b (0.433 ns, the same `N = 20` system at one thread) the row is about 1.9 × 10^11 times rho, and with one inversion costed as one multiplication (`M = 4`) 1.0 x 10^13 times. It is a lower bound (one solve per relation, no failed targets, no factor-base build, no linear algebra) and counts 925,571 unpriced operations per system, which do not move it at any `w`. It is a second priced size, but no scaling fit is claimed: the two ratios to rho, 1.9 x 10^9 at `N = 20` and 7.7 × 10^10 at `N = 24`, are 40 times apart for 4 more unknowns. **P12 held** (at least 10^5 times rho). The distance that remains to rho is the §3.3 table plus this row; nothing in this addendum changes any ecbench arm. Tables: `tables/f4_priced.md`, `tables/ecbench_arms.md`; graph:



ecbench sessions (measured) and the derived F4 lower bound (PROTOCOL 4.4). Solid: Solinas, dashed: Proth.

8.6 Predictions against outcomes (PROTOCOL §12.4)

	prediction	outcome
P8	every equality check holds, counts bit-identical	**held** (E1, E2, E3, §8.3)
P9	fixed peak at most 0.90 x pre-fix at `N = 22`	**failed** (0.976; 1.12 at `N = 20`)
P10	R20c finishes, `D` 6 or 7, peak 5.5-7.5 GB, within 5 h	**held** (`D = 6`, 5.99 GB, 2.0 h)
P11	at least half of `B'` non-zero at the largest steps	**held at step 27 (57%), failed at steps 23 and 26 (41%, 35%)**
P12	F4 arm at least 10^5 x rho at `N = 24`	**held** (7.7 × 10^10)

8.7 Deviations, failures, limits

- **E1 was started once before the amendment was committed** (20:48:41 UTC) and stopped within a minute with no output read; its three files were deleted and it was re-run after the commit (`cf444326b`).
- **E2's equality** is fixed against pre-fix at the same thread count (2); the frozen rows are 1-thread, so the amendment's wording ("E1's equality") could not apply to them. R20c's

multiply-adds equal R18b's at the steps they share (2 against 4 threads), where the amendment expected a different chunking.

- **Memory figures are one run per arm** on a shared host (load average 4-7, other processes' compiles and tests running); wall times are descriptive. E2's `N = 22` wall times (1,718 s, 1,693 s) are not a speed comparison.
- **The kill itself** was another process's allocation (Amendment 2); that diagnosis is unchanged. What §8.2 adds is that the footprint at the kill was the run's own peak.
- **Scope.** `N = 24` is 11% of P-224's `N` (§2, "Scope"); every statement about deeper `N` is extrapolation and is not made. `m = 3, N = 21` and any larger `m` cell were not run. Nothing here attacks P-224.
- **Binaries.** Fixed: `pkm_tower_pilot` sha256 `e731c65114686cfe...` built from `cf444326b`; pre-fix: `803fae9946b0154d...`, built from the merged tree before these changes (`followup_20261022/BINARY.sha256`). rustc 1.97.0; 4 vCPU Xeon, 15 GiB, shared.

8.8 Reproduce

```
        export CARGO_TARGET_DIR=/path/to/target
        cargo build --release --example pkm_tower_pilot --example pkm_tower_check --example
pkm_p224_shape_report
        BIN=... BASE=...
research/pkm_p224_kummer_shape_20261022/followup_20261022/run_followup.sh E1   # then E2,
E3, R20c
        followup_20261022/check_rows.sh NEW.jsonl FROZEN.jsonl;
followup_20261022/check_steps.sh NEW.log OLD.log
        pkm_p224_shape_report f4 followup_20261022/merged_runs followup_20261022
        pkm_p224_shape_report price followup_20261022/merged_runs followup_20261022
curves.jsonl sessions/s20 sessions/s24 sessions/s26 sessions/s28
        pkm_p224_shape_report mem followup_20261022 runs/R18b-SL24-m2-N24-sizing.log
followup_20261022/runs/R20c0-SL24-m2-N24.steps.jsonl followup_20261022/runs/R20c-SL24-m2-
N24.steps.jsonl
        research/pkm_p224_kummer_shape_20261022/render.sh
```